Optimized Test Automation Framework Overview

🎯 1. Introduction: A Foundation for Quality

This document outlines the design and architecture of the test automation framework built for the OpenCart project.

The framework is engineered for:

- **Maintainability**
- **Reusability**
- **Scalability**

It strictly follows the **Page Object Model (POM)** design pattern, separating test logic from page interaction code.

🛠️ 2. Technology Stack & Environment

Component	Technology	Role
Programming Language	**Java**	Core logic implementation.
Web Automation	**Selenium WebDriver**	Browser interaction and control.
Test Runner	**TestNG**	Test execution management and reporting.
Build Tool	Maven	Dependency management and build lifecycle.
Browser Target	Google Chrome	Primary execution environment.

🏗️ 3. Layered Architecture

The framework utilizes a three-tiered layered architecture to ensure clear separation of concerns:

A. Test Layer

- **Location**: src/test/java/Tests
- **Role**: Contains the executable test scripts (e.g., CartPageTest, LoginPageTest).
- **Function**: These classes extend BaseTestClass to inherit all necessary setup and teardown routines.

B. Page Layer (Page Object Model)

- **Location**: src/test/java/Pages
- **Role**: Defines the Page Objects (e.g., HomePage, CartPage).
- **Function**: Encapsulates web element locators and the methods for interacting with them, abstracting the UI details from the tests.

C. Base Layer (Core Setup)

- **Location**: src/test/java/BaseTest
- **Role**: Contains the BaseTestClass.
- **Function**: Handles critical operations like **WebDriver initialization** (ChromeDriver), global configuration (e.g., implicit waits, window maximization), and universal test teardown.

📂 4. Project Folder Structure

A standard, clean structure is maintained for quick navigation and clear organization:

```
src/
└── test/
    └── java/
        ├── BaseTest/
        │   └── BaseTestClass.java  # Driver setup & teardown logic
        ├── Pages/
        │   ├── HomePage.java       # Page Object classes
        │   ├── CartPage.java
        │   └── LoginPage.java
        └── Tests/
            ├── CartPageTest.java   # Executable Test Scripts
            ├── HomeTest.java
            └── LoginPageTest.java
```

✨ 5. Key Features & Benefits

Feature	Description	Benefit

Modularity (POM)	Logic is cleanly separated between Test scripts and Page Objects.	Easier code reading and maintenance.
Reusability	Actions (e.g., clickLoginButton(), enterUsername()) are defined once.	Reduces code duplication and speeds up development.
Robustness	Employs WebDriverWait (Explicit Waits).	Reduces test flakiness by gracefully handling dynamic element loading.
Reporting	Leverages TestNG's built-in capabilities.	Provides detailed execution logs and comprehensive HTML reports.

⚙️ 6. Execution Flow Summary

The test execution follows a predictable and reliable three-step cycle:

1. **Setup Phase**
   - The BaseTestClass initializes the ChromeDriver.
   - The browser navigates to the application URL: https://demoblaze.com/.
2. **Test Execution**
   - Test methods call Page Objects (Pages/ classes).
   - Page Objects perform the necessary UI actions (clicks, inputs).
   - The test script asserts the results.
3. **Teardown Phase**
   - The driver.quit() method is executed to close the browser session and clean up resources.